TravelGo — Project Documentation & Interview Guide

Project Title: TravelGo Smart Backpack Landing Page

Author / Developer: Swayam

GitHub Repository: github.com/swayam200545/TravelGo

Vercel Deployment: travelgo.vercel.app

Tech Stack: HTML5, CSS3, Vanilla JavaScript (ES6+)

Date: October 2026

1. Executive Summary & Technical Approach

TravelGo is a high-converting, responsive landing page engineered for a premium smart travel backpack brand. The core engineering philosophy focused on creating a lightning-fast, framework-free web application with zero external dependencies, leveraging native modern web standards to deliver high visual polish and smooth user interactions.

Core Objectives & Architectural Highlights:

- **Zero-Dependency Performance:** Built entirely with native HTML5, CSS3, and ES6+ JavaScript. Without the overhead of heavy JavaScript framework bundles, the initial page payload is minimal and loads instantly.
- **Dual-Way State Synchronization:** Synchronizes model selections (`TravelGo Lite`, `TravelGo Pro`, `TravelGo Max`) bidirectionally between the main product customizer and the express checkout form.
- **Interactive Checkout & Mock Payment Gateway:** Features a 3-step express checkout workflow with tabbed mock payment methods (UPI/QR, Credit/Debit Cards, Net Banking, COD), promo coupon handler (`TRAVEL15`), live Indian Rupee (`₹`) price calculation, and simulated payment confirmation.
- **Photorealistic AI Visual Asset Strategy:** Utilized AI image generation prompts to craft ultra-realistic 8K studio photography for product variants, 180° lay-flat organization, hydrophobic fabric close-ups, and airport lifestyle imagery.
- **SEO & Accessibility (a11y) Standards:** Adheres to semantic HTML5 structure (`
`, `
`, `
`, `
`, `

▶ Details

2. System Architecture & Component Breakdown

Component / File	Role & Key Responsibilities
`index.html`	Contains the semantic layout across 8 sections: Announcement Bar, Sticky Header, Hero Banner, Visual Showcase, Interactive Product Selector, Features Grid, Comparison Table, Customer Reviews, FAQ Accordion, Checkout & Payment Section, and Order Confirmation Modal.
`css/style.css`	Defines the visual design system using CSS custom properties (`:root` tokens), CSS Grid, Flexbox, glassmorphism (`backdrop-filter: blur()`), glowing gradients, print styles, and responsive media query breakpoints (`1024px`, `860px`, `540px`).
`js/script.js`	Houses the application state (`selectedKey`, `quantity`, `currentDiscountPercent`, `selectedPaymentMode`), handles event listeners for variant buttons and radio cards, computes live totals, formats currency, validates form inputs, and manages modal popups.

Key JavaScript Logic & Functions Walkthrough:

1. `selectVariant(key)`: The centralized controller for product selection. When triggered (either by clicking a customizer button or selecting a checkout radio card), it updates `selectedKey`, toggles active CSS classes, initiates a smooth image fade transition (`opacity: 0 -> opacity: 1`), and triggers `updateCalculatedTotals()`.

```
function selectVariant(key) {
  if (!products[key]) return;
  selectedKey = key;

  // 1. Update Main Product Customizer UI
  variantButtons.forEach(btn => {
    const active = btn.dataset.variant === key;
    btn.classList.toggle("active", active);
    btn.setAttribute("aria-checked", active);
  });

  // 2. Update Checkout Section Option Cards
  Object.keys(checkoutProductCards).forEach(k => {
    if (checkoutProductCards[k]) {
      const active = k === key;
      checkoutProductCards[k].classList.toggle("active", active);
      const radioInput = checkoutProductCards[k].querySelector('input[type="radio"]');
      if (radioInput) radioInput.checked = active;
    }
```

```
  });

  updateProductDetails();
  updateCalculatedTotals();
}
```

2. `updateCalculatedTotals()`: Computes base subtotal (`price * quantity`), applies promo discounts (`baseSubtotal * currentDiscountPercent / 100`), formats numbers into Indian Rupee strings using `toLocaleString("en-IN")`, and updates DOM elements in both the product summary and checkout receipt.

3. `validateCheckoutForm()` & Input Sanitization: Performs client-side validation using standard regex patterns for Indian mobile numbers (`/^(\+91|91|0)?[6–9]\d{9}$/`), emails, and 6–digit postal pincodes. Highlights invalid fields with `.invalid` borders and inline error prompts.

3. Master Interview Guide: Top 10 Technical Questions & Answers

Q1. Why did you build this application using Vanilla JavaScript instead of a framework like React or Next.js?

Answer: For a landing page, performance and instant load time are paramount for conversion. React or Next.js introduce virtual DOM overhead, bundle hydration costs, and extra JavaScript payload size (often 100KB+ gzipped). By using Vanilla ES6+ JS, the application executes natively in the browser with 0KB external dependencies, achieving a near-100 Google Lighthouse performance score. It also demonstrates strong foundational mastery of native DOM manipulation, event delegation, and state management.

Q2. How did you structure state management without Redux or Context API?

Answer: I established a single-source-of-truth state using modular global variables (`selectedKey`, `quantity`, `currentDiscountPercent`, `selectedPaymentMode`) paired with a dictionary data object (`products`). Whenever state updates occur, dedicated observer functions (`updateProductDetails()`, `updateCalculatedTotals()`) run to update all dependent DOM views synchronously across both the main product section and checkout card.

Q3. How does the product synchronization work between the hero/customizer section and the checkout section?

Answer: I unified the selection logic into a single function `selectVariant(key)`. Clicking a variant button in the product customizer or checking a radio card in the checkout section both invoke `selectVariant(key)`. This function updates the active CSS class and `checked` attribute of both sections simultaneously, ensuring seamless alignment across the entire page.

Q4. How did you implement responsive design and test across different screen sizes?

Answer: I implemented a desktop-first responsive strategy using CSS Grid and Flexbox with media breakpoints at `1024px` (tablets/laptops), `860px` (portrait tablets/mobile navigation toggle), and `540px` (small smartphones). On mobile screens, multi-column layouts stack into single-column cards, the desktop navigation transforms into a slide-down hamburger menu, and a fixed bottom mobile sticky bar appears for easy 1-tap checkout.

Q5. How does the mock payment gateway workflow operate?

Answer: The payment UI features a 4-tab container allowing users to select between UPI/QR, Credit/Debit Card, Net Banking, and Cash on Delivery. Upon submitting valid checkout details, the submit button triggers an asynchronous loading simulation (`setTimeout` for 1.2 seconds) displaying "🔒 *Processing Payment...*"before launching an Order Confirmation Modal (`#orderModal`) populated with a dynamically generated Order ID (`#TGO-XXXXX`), payment method name, address, and receipt summary.

Q6. What visual design techniques were used to create a modern aesthetic?

Answer: I designed a cohesive color palette based on deep slate (`#0F172A`) and royal sapphire blue (`#2563EB`). Visual depth was achieved using backdrop-filter glassmorphism (`backdrop-filter: blur(12px)` on navigation and sticky bars), subtle box shadows (`0 10px 30px rgba(15,23,42,0.08)`), glowing radial gradients, floating key feature chips with CSS floating keyframe animations, and custom typography pairing Sora with Plus Jakarta Sans.

Q7. How did you handle form validation and user feedback?

Answer: I implemented real-time and submit-time validation. Upon submission, `validateCheckoutForm()` tests each field against specific rules (e.g., regex pattern `/^[6-9]\d{9}$/` for 10-digit Indian mobile numbers and `/^\d{6}$/` for pincodes). If invalid, `.invalid` CSS classes highlight the field, error text appears in a dedicated alert span, and focus shifts to the first failing input. Input event listeners automatically clear error states as the user types.

Q8. How is SEO and Web Accessibility (a11y) implemented?

Answer: SEO is optimized using descriptive `